

Python Interview Mastery

FAANG · Bay Area · Enterprise

Version 1.0 — Beginner · Intermediate · Advanced

What's inside

- 180+ questions across 15 topic areas
- Three difficulty tiers — Beginner, Intermediate, Advanced
 - Detailed answers with code examples
 - Trick notes, gotchas, and pitfalls interviewers love
- Big-O cheatsheet · Algorithm pattern templates · Sort comparisons
- FAANG company-specific problems (Google, Meta, Amazon, Apple, Netflix)
 - Bay Area enterprise picks (Zoom, Cisco, Oracle, Palo Alto, Zscaler)
 - ASCII diagrams for visual learners
 - Pythonic idioms and best practices

BEGINNER

INTERMEDIATE

ADVANCED

Table of Contents

1.	Python Fundamentals	B:12 I:7 A:2 · 21 total
2.	Data Structures	B:4 I:5 A:2 · 11 total
3.	Algorithms & Problem Patterns	B:3 I:5 A:4 · 12 total
4.	OOP & Design Patterns	B:3 I:4 A:2 · 9 total
5.	Decorators, Generators, Iterators	B:1 I:3 A:3 · 7 total
6.	Concurrency & Async	B:1 I:3 A:2 · 6 total
7.	Python Internals & Gotchas	B:2 I:3 A:3 · 8 total
8.	Exceptions & Error Handling	B:2 I:3 A:1 · 6 total
9.	Strings & Regex	B:2 I:3 A:1 · 6 total
10.	FAANG Coding Problems	B:0 I:5 A:6 · 11 total
11.	Pythonic Idioms & Best Practices	B:2 I:2 A:2 · 6 total
12.	Python in System Design	B:0 I:2 A:3 · 5 total
13.	Testing & Debugging	B:2 I:3 A:2 · 7 total
14.	Standard Library Essentials	B:1 I:3 A:2 · 6 total
15.	Company-Favorite Questions	B:0 I:5 A:7 · 12 total
16.	Cheatsheets & Reference Tables	<i>5 reference sheets</i>

Total: 133 questions · Beginner: 35 · Intermediate: 56 · Advanced: 42

1. Python Fundamentals

B · 12

I · 7

A · 2

Q1. What are Python's key features that make it popular?

BEGINNER

ANSWER

Python is interpreted (no compile step), dynamically typed (types checked at runtime), strongly typed (no implicit conversions like '1' + 1), high-level, garbage-collected, has a huge standard library ('batteries included'), and supports multiple paradigms — OOP, functional, procedural. It's also platform-independent and has massive community/ecosystem (PyPI has 500K+ packages).

Q2. Mutable vs Immutable types — give examples and explain why it matters.

BEGINNER

ANSWER

Mutable: list, dict, set, bytearray — can be changed in place. Immutable: int, float, str, tuple, frozenset, bytes — cannot change; any 'modification' creates a new object. It matters because: (1) immutable objects are hashable (can be dict keys), (2) mutable defaults in function args cause bugs, (3) passing mutable objects to functions can cause side effects.

```
>>> a = [1, 2, 3]
>>> b = a
>>> b.append(4)
>>> a
[1, 2, 3, 4]    # a changed too – same object

>>> s = 'hello'
>>> s[0] = 'H'    # TypeError: str is immutable
```

■ **TRICK:** tuples are immutable but can **CONTAIN** mutable objects. `t = ([1,2], 3); t[0].append(4)` works! The tuple's references don't change, but the list inside does.

Q3. What's the difference between `is` and `==`?

BEGINNER

ANSWER

`==` compares VALUES (calls `__eq__`). `is` compares IDENTITY (same memory address, same object). Use `is` only for None, True, False, and singletons. Common gotcha: small int caching — Python caches ints from -5 to 256, so 'a is b' may be True for small ints but False for large ones.

```
>>> a = 256; b = 256
>>> a is b
True
>>> a = 257; b = 257
>>> a is b
False    # NOT cached!

>>> x = None
>>> x is None    # ✓ correct
>>> x == None    # works but bad style
```

Q4. Explain Python's data types and their typical use cases.

BEGINNER

ANSWER

Numbers: int (arbitrary precision), float (64-bit), complex. Sequences: list (mutable, ordered), tuple (immutable, ordered), range, str. Mappings: dict (key-value, hash-based, ordered since 3.7). Sets: set (mutable, unique), frozenset (immutable). Binary: bytes, bytearray, memoryview. Boolean: bool (subclass of int — True==1, False==0). None: the null singleton.

Q5. What is PEP 8 and why does it matter in interviews?

BEGINNER

ANSWER

PEP 8 is Python's official style guide: 4-space indent, 79-char lines (or 99 modern), snake_case for vars/functions, PascalCase for classes, UPPER_CASE for constants, two blank lines between top-level defs, import order (stdlib, third-party, local). Interviewers notice — clean code signals professionalism. Tools: black (formatter), flake8/ruff (linter), isort (imports).

Q6. List vs Tuple — when do you use each?

BEGINNER

ANSWER

Lists are mutable, dynamic, slightly more memory. Tuples are immutable, hashable (usable as dict keys/set elements), faster to create, and signal 'fixed collection' or heterogeneous record. Rule: use tuple when contents won't change (coordinates, RGB, function returning multiple values); list for collections you'll modify.

■ **INTERVIEW TIP:** tuples have only `count()` and `index()` — that's it. Lists have `append`, `extend`, `insert`, `remove`, `pop`, `clear`, `sort`, `reverse`, `copy`.

Q7. How does Python handle memory management?

BEGINNER

ANSWER

Python uses (1) reference counting — every object tracks how many references point to it; when count hits 0, memory is freed immediately. (2) Generational garbage collector — handles cyclic references (a -> b -> a) that reference counting alone can't free. Memory pools (PyMalloc) optimize small object allocation. You can inspect with `sys.getrefcount()` and `gc` module.

DIAGRAM

```
ref count diagram:
x = [1,2,3]      # list obj refcount = 1
y = x           # refcount = 2
del x           # refcount = 1
y = None        # refcount = 0 → freed
```

Q8. What are *args and **kwargs?

BEGINNER

ANSWER

*args collects extra positional args into a tuple. **kwargs collects extra keyword args into a dict. Names are convention — *a, **kw work too. Order in signature: positional, *args, keyword-only, **kwargs. Useful for wrappers/decorators that forward arbitrary args.

```
def f(a, *args, b=10, **kwargs):
    print(a, args, b, kwargs)

f(1, 2, 3, b=20, x=99)
# 1 (2, 3) 20 {'x': 99}
```

Q9. Explain Python's truthiness rules.

BEGINNER

ANSWER

Falsy: False, None, 0, 0.0, "", [], {}, (), set(), and objects whose `__bool__` returns False. Everything else is truthy. This enables idioms like 'if my_list:' (checks non-empty). Override with `__bool__` (Python 3) or `__len__` (falls back if `__bool__` undefined).

■ **PITFALL:** 'if x:' is NOT the same as 'if x is not None:'. The first treats 0, "", [] as None — bug if x=0 is valid input. Be explicit when None matters.

Q10. What does the `pass` statement do?

BEGINNER

ANSWER

It's a no-op — does nothing. Used as placeholder where syntax requires a statement but you have no logic yet (empty function, class, except block). Different from `continue`` (skip to next loop iteration) and `...`` (Ellipsis literal, often used in type stubs).

Q11. How do you swap two variables without a temp variable?

BEGINNER

ANSWER

a, b = b, a — tuple packing/unpacking. Python evaluates the right side fully first (creates tuple), then unpacks to left. Works for any number of variables. No XOR trick needed (and XOR doesn't work on Python ints reliably anyway due to arbitrary precision).

Q12. What's the difference between `range()` and `xrange()`?

BEGINNER

ANSWER

In Python 2, `range()` returned a list; `xrange()` returned an iterator (lazy). In Python 3, `xrange()` is gone — `range()` is the lazy iterator. To get a list, do `list(range(n))`. Memory matters: `range(10**9)` is fine; `list(range(10**9))` would crash.

Q13. Explain shallow vs deep copy with an example.

INTERMEDIATE

ANSWER

Shallow copy (`copy.copy` or `list.copy()`) creates a new outer container but shares inner references. Deep copy (`copy.deepcopy`) recursively copies everything. For lists of immutables, shallow is enough. For nested mutables, shallow can cause hidden mutations.

```
import copy
x = [[1, 2], [3, 4]]
y = copy.copy(x)          # shallow
y[0].append(99)
print(x)  # [[1, 2, 99], [3, 4]] ← x mutated!

z = copy.deepcopy(x)
z[0].append(0)
print(x)  # unchanged
```

Q14. What is the GIL and why does it exist?

INTERMEDIATE

ANSWER

The Global Interpreter Lock is a mutex that allows only ONE thread to execute Python bytecode at a time within a single process. It exists because CPython's memory management (reference counting) isn't thread-safe — without GIL, two threads could decrement a refcount simultaneously, corrupting memory. Impact: threading is great for I/O-bound work (releases GIL during I/O) but useless for CPU-bound parallelism. For CPU-bound, use multiprocessing, asyncio, C extensions, or numpy.

DIAGRAM

```
GIL flow:
[Thread A]■■■■
[Thread B]■■■■■■[GIL]■■■[Python bytecode]
[Thread C]■■■■      ↑ only one at a time

I/O releases GIL → threads CAN overlap I/O waits
```

Q15. What's the difference between `__init__` and `__new__`?

INTERMEDIATE

ANSWER

`__new__` creates the instance (returns it). `__init__` initializes the already-created instance (returns None). `__new__` is rarely overridden — needed for immutable types (str, tuple, int) and for singleton/metaclass patterns. Order: `__new__` → `__init__`.

```
class Singleton:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
```

Q16. Explain method resolution order (MRO) and the C3 linearization.

INTERMEDIATE

ANSWER

MRO is the order Python searches base classes for an attribute. Python 3 uses C3 linearization, which guarantees: (1) subclasses before parents, (2) parent declaration order preserved, (3) consistent across the hierarchy. Inspect with `ClassName.__mro__` or `ClassName.mro()`. Used to resolve 'diamond' inheritance.

```
class A: pass
class B(A): pass
class C(A): pass
class D(B, C): pass
D.__mro__
# (D, B, C, A, object)
# ↑ diamond resolved: D→B→C→A→object
```

Q17. What is duck typing? Give an example.

INTERMEDIATE

ANSWER

'If it walks like a duck and quacks like a duck, it's a duck.' Python cares about behavior (methods, attributes) not type. You don't check `isinstance` — you try and handle exceptions (EAFP: Easier to Ask Forgiveness than Permission). Enables flexibility but can hide bugs. Modern Python adds `typing.Protocol` for structural subtyping (typed duck typing).

Q18. What's the difference between `@staticmethod`, `@classmethod`, and instance methods?

INTERMEDIATE

ANSWER

Instance method (default): first arg is `self` — operates on the instance. `classmethod`: first arg is `cls` — operates on the class itself, often used for alternative constructors. `staticmethod`: no implicit first arg — just a function namespaced inside a class. Choose based on what the method needs to access.

```
class Date:
    def __init__(self, y, m, d): self.y, self.m, self.d = y, m, d

    @classmethod
    def from_string(cls, s):      # alt constructor
        y, m, d = map(int, s.split('-'))
        return cls(y, m, d)

    @staticmethod
    def is_leap(year):            # pure utility
        return year % 4 == 0 and (year % 100 != 0 or year % 400 == 0)
```

Q19. How does Python's `with` statement work?

INTERMEDIATE

ANSWER

with` is the context manager protocol — guarantees cleanup. The object must implement `__enter__` (returns the resource) and `__exit__` (cleanup, even on exception). Common uses: file I/O, locks, DB connections, opening sessions. `contextlib.contextmanager` decorator turns a generator into a context manager.

```
from contextlib import contextmanager

@contextmanager
def timer():
    import time
    t = time.time()
    yield # body runs here
    print(f'{time.time()-t:.2f}s')

with timer():
    sum(range(10**6))
```

Q20. Walk me through Python's memory model and object internals.

ADVANCED

ANSWER

Every Python object has a PyObject header containing: refcount (ssize_t), type pointer (*PyTypeObject), and type-specific data. Small ints (-5..256) and short strings are cached/interned. Lists are arrays of PyObject pointers with over-allocation (~12.5% growth). Dicts are open-addressing hash tables (since 3.6: compact, ordered). Use `sys.getsizeof()` to inspect.

DIAGRAM

```
PyObject layout:  
■■■■■■■■■■■■■■■■■■■■  
■ ob_refcnt          ■ 8 bytes  
■■■■■■■■■■■■■■■■■■■■  
■ ob_type *         ■ 8 bytes  
■■■■■■■■■■■■■■■■■■■■  
■ type-specific...■  
■■■■■■■■■■■■■■■■■■■■
```

```
List growth: 0, 4, 8, 16, 25, 35, 46, 58...
List of 1000 ints ≈ 8KB on 64-bit
```


Q21. What are descriptors? When have you used them?

ADVANCED

ANSWER

A descriptor is any object implementing `__get__`, `__set__`, and/or `__delete__`. They control attribute access on classes. Data descriptors (have `__set__`) take precedence over instance `__dict__`; non-data only override if attribute not in instance `__dict__`. Used by: `@property`, `@classmethod`, `@staticmethod`, ORMs (django Model fields), slots, type validation.

```
class Positive:
    def __set_name__(self, owner, name): self.name = name
    def __get__(self, obj, type=None):
        return obj.__dict__.get(self.name)
    def __set__(self, obj, value):
        if value < 0: raise ValueError
        obj.__dict__[self.name] = value

class Order:
    qty = Positive()

o = Order(); o.qty = 5      # ok
o.qty = -1                 # ValueError
```

2. Data Structures

B · 4

I · 5

A · 2

Q22. What's the time complexity of common list operations?

BEGINNER

ANSWER

Append $O(1)$ amortized, `pop()` from end $O(1)$, `pop(0)` $O(n)$, `insert(i,x)` $O(n)$, index lookup $O(1)$, `'in'` $O(n)$, sort $O(n \log n)$, reverse $O(n)$, copy $O(n)$, len $O(1)$. Lists are dynamic arrays — random access is fast, but inserting/deleting at the front is slow. Use `collections.deque` for $O(1)$ front operations.

DIAGRAM

```
List vs Deque (head ops):
list.pop(0)    →  $O(n)$     ← shifts everything left
deque.popleft() →  $O(1)$ 

list.insert(0, x) →  $O(n)$ 
deque.appendleft(x) →  $O(1)$ 
```

Q23. Time complexity of dict and set operations?

BEGINNER

ANSWER

Average: `insert/get/delete/'in'` are $O(1)$ — hash table. Worst case (many collisions): $O(n)$ — extremely rare with Python's hash. Iteration $O(n)$. Memory: $\sim 3x$ more than equivalent list due to hash table overhead. `dict.keys()/values()/items()` return views ($O(1)$, live).

Q24. What is a set and when should you use one?

BEGINNER

ANSWER

Unordered collection of unique, hashable elements. Use for: membership testing ($O(1)$), deduplication (`set(list)`), set algebra (union `|`, intersection `&`, difference `-`, symmetric_diff `^`). Doesn't support indexing. `frozenset` is the immutable, hashable variant.

```
a = {1, 2, 3}
b = {2, 3, 4}
a | b    # {1, 2, 3, 4}    union
a & b    # {2, 3}         intersection
a - b    # {1}            difference
a ^ b    # {1, 4}         symmetric difference
```

Q25. How do you iterate over a dict?

BEGINNER

ANSWER

Default iteration is over keys. Use `.items()` for (key, value), `.values()` for values, `.keys()` for explicit keys. As of Python 3.7+, dict insertion order is guaranteed preserved.

```
d = {'a': 1, 'b': 2}
for k in d: ...           # keys
for v in d.values(): ...
for k, v in d.items(): ...
```

Q26. Explain Python's collections module — what's in it?

INTERMEDIATE

ANSWER

`OrderedDict` (pre-3.7 ordered dict; now mainly for `move_to_end/equality` semantics), `defaultdict` (auto-creates missing keys), `Counter` (multiset/histogram), `deque` (double-ended queue, $O(1)$ both ends), `namedtuple` (lightweight class), `ChainMap` (view multiple dicts as one), `UserDict`/`UserList`/`UserString` (subclassable wrappers).

```
from collections import Counter, defaultdict, deque

Counter('mississippi').most_common(2) # [('i', 4), ('s', 4)]

graph = defaultdict(list)
graph['A'].append('B')    # no KeyError

q = deque([1, 2, 3])
q.appendleft(0); q.pop()  #  $O(1)$  both ends
```

Q27. How would you implement a stack and a queue in Python?

INTERMEDIATE

ANSWER

Stack (LIFO): use a list with `append()` and `pop()` — both $O(1)$. Queue (FIFO): NEVER use `list.pop(0)` — it's $O(n)$. Use `collections.deque` with `append()` and `popleft()` — both $O(1)$. For thread-safe queues, use `queue.Queue` (LIFO: `queue.LifoQueue`, priority: `queue.PriorityQueue`).

```
# Stack
stack = []
stack.append(1); stack.append(2)
stack.pop()    # 2

# Queue
from collections import deque
q = deque()
q.append(1); q.append(2)
q.popleft()    # 1
```

Q28. How is a Python dict implemented internally?

INTERMEDIATE

ANSWER

Open-addressing hash table. Since 3.6, it's 'compact': a dense array of entries + a sparse index array. Insertion order preserved. Load factor $\sim 2/3$ triggers resize (doubles). Hash collisions resolved via probing. Keys must be hashable (`__hash__` and `__eq__`). `__hash__` should be consistent with `__eq__`: equal objects must have equal hashes.

DIAGRAM

```
Compact dict (3.6+):
indices: [-, 1, -, 0, -, 2, -, -]    # sparse
entries: [(hash, 'a', 1),             # dense
          (hash, 'b', 2),
          (hash, 'c', 3)]
→ preserves insertion order, less memory
```

Q29. What is a heap and how do you use heapq?

INTERMEDIATE

ANSWER

A heap is a complete binary tree where `parent <= children` (min-heap). `heapq` is Python's min-heap on a list. `heappush` $O(\log n)$, `heappop` $O(\log n)$, `heapify` $O(n)$. For max-heap, negate values. Used for: top-K problems, scheduling, Dijkstra's algorithm, median finding (two heaps).

```
import heapq
h = [5, 1, 8, 3]
heapq.heapify(h)           # in-place O(n)
heapq.heappush(h, 2)
heapq.heappop(h)           # 1
heapq.nsmallest(2, h)      # 2 smallest
heapq.nlargest(3, h)
```

Q30. Implement a LRU cache without using `functools.lru_cache`.

INTERMEDIATE

ANSWER

Use `OrderedDict`: move accessed keys to end, `popitem(last=False)` evicts the oldest. Or doubly-linked list + dict for explicit control. The dict gives $O(1)$ lookup; the list/order tracks recency.

```
from collections import OrderedDict

class LRU:
    def __init__(self, cap):
        self.cap = cap
        self.d = OrderedDict()

    def get(self, k):
        if k not in self.d: return -1
        self.d.move_to_end(k)
        return self.d[k]

    def put(self, k, v):
        if k in self.d: self.d.move_to_end(k)
        self.d[k] = v
        if len(self.d) > self.cap:
            self.d.popitem(last=False)
```

Q31. Implement a Trie (prefix tree).

ADVANCED

ANSWER

Tree where each node represents a character. Common use: autocomplete, spell-check, IP routing. Insert/search/prefix all $O(m)$ where m = word length. Standard implementation uses dict children + end-of-word flag.

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.end = False

class Trie:
    def __init__(self): self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for c in word:
            if c not in node.children:
                node.children[c] = TrieNode()
            node = node.children[c]
        node.end = True

    def search(self, word):
        node = self._walk(word)
        return node is not None and node.end

    def starts_with(self, prefix):
        return self._walk(prefix) is not None

    def _walk(self, s):
        node = self.root
        for c in s:
            if c not in node.children: return None
            node = node.children[c]
        return node
```

Q32. How would you implement Union-Find (Disjoint Set Union)?

ADVANCED

ANSWER

Track connected components. Path compression + union by rank gives near- $O(1)$ amortized (inverse Ackermann). Used for Kruskal's MST, cycle detection, connected components in graphs.

```
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x]) # path compression
        return self.parent[x]

    def union(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py: return False
        if self.rank[px] < self.rank[py]: px, py = py, px
        self.parent[py] = px
        if self.rank[px] == self.rank[py]: self.rank[px] += 1
        return True
```

3. Algorithms & Problem Patterns

B · 3

I · 5

A · 4

Q33. What is Big-O notation? Explain common complexities.

BEGINNER

ANSWER

Big-O describes the worst-case growth rate of time/space as input size n grows. Common: $O(1)$ constant, $O(\log n)$ logarithmic (binary search), $O(n)$ linear (single loop), $O(n \log n)$ linearithmic (efficient sorts), $O(n^2)$ quadratic (nested loops), $O(2^n)$ exponential (subsets), $O(n!)$ factorial (permutations). Drop constants and lower terms.

DIAGRAM

```
Growth (n=100):
O(1)      = 1
O(log n)  = 7
O(n)      = 100
O(n log n) = 664
O(n^2)    = 10,000
O(2^n)    = 1.27×10^30 ← avoid for n>~25
```

Q34. Implement binary search.

BEGINNER

ANSWER

Classic divide-and-conquer on a sorted array. $O(\log n)$. Off-by-one is the #1 bug — be careful with lo/hi boundaries and mid calculation. Use bisect module in Python for production.

```
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target: return mid
        elif arr[mid] < target: lo = mid + 1
        else: hi = mid - 1
    return -1

# Built-in:
import bisect
bisect.bisect_left(arr, target)  # insertion point
```

■ **TEMPLATE:** when stuck on binary search variants, use this invariant: 'arr[lo..hi] contains the answer'. Adjust based on whether you want first/last occurrence, smallest $\geq$, etc.

Q35. Two Sum — find indices of two numbers that add up to target.

BEGINNER

ANSWER

Classic. Brute force $O(n^2)$. Better: one pass with hash map. For each x, check if (target - x) was seen.

```
def two_sum(nums, target):
    seen = {}  # value → index
    for i, x in enumerate(nums):
        if target - x in seen:
            return [seen[target - x], i]
        seen[x] = i
    return []
```

Q36. Explain the Two Pointers pattern with an example.

INTERMEDIATE

ANSWER

Use two indices moving through the array, often from both ends or at different speeds. Saves a nested loop, turning $O(n^2)$ into $O(n)$. Works on sorted arrays, palindromes, partition problems.

```
# Sorted array, find pair sum = target
def pair_sum(arr, target):
    l, r = 0, len(arr) - 1
    while l < r:
        s = arr[l] + arr[r]
        if s == target: return (l, r)
        elif s < target: l += 1
        else: r -= 1
    return None

# Palindrome check
def is_pal(s):
    l, r = 0, len(s) - 1
    while l < r:
        if s[l] != s[r]: return False
        l += 1; r -= 1
    return True
```

Q37. Explain Sliding Window with an example.

INTERMEDIATE

ANSWER

Maintain a window $[l, r]$ over the array. Expand r , shrink l based on a condition. Used for substring/subarray problems on contiguous ranges. Fixed-size (e.g. max sum of k elements) or variable-size (e.g. longest substring without repeating).

```
# Longest substring without repeating chars
def longest(s):
    last = {}
    l = 0; best = 0
    for r, c in enumerate(s):
        if c in last and last[c] >= l:
            l = last[c] + 1
        last[c] = r
        best = max(best, r - l + 1)
    return best
# 'abcabcbb' → 3 ('abc')
```


Q38. BFS vs DFS — when do you use each?

INTERMEDIATE

ANSWER

BFS (queue, level-by-level): shortest path in unweighted graph, level-order tree traversal, finding nearest target. DFS (stack/recursion, go deep): cycle detection, topological sort, connectivity, path existence, all paths, backtracking. BFS uses more memory (whole level); DFS uses less but can blow stack on deep graphs.

```
from collections import deque

def bfs(graph, start):
    visited = {start}; q = deque([start])
    while q:
        node = q.popleft()
        for nei in graph[node]:
            if nei not in visited:
                visited.add(nei); q.append(nei)
    return visited

def dfs(graph, start, visited=None):
    if visited is None: visited = set()
    visited.add(start)
    for nei in graph[start]:
        if nei not in visited: dfs(graph, nei, visited)
    return visited
```

Q39. Reverse a linked list.

INTERMEDIATE

ANSWER

Iterate, repointing each node's next pointer to the previous one. $O(n)$ time, $O(1)$ space. Recursive version is elegant but $O(n)$ stack.

```
class Node:
    def __init__(self, v, nxt=None): self.v, self.next = v, nxt

def reverse(head):
    prev = None
    cur = head
    while cur:
        nxt = cur.next
        cur.next = prev
        prev = cur
        cur = nxt
    return prev
```

Q40. Detect a cycle in a linked list (Floyd's algorithm).

INTERMEDIATE

ANSWER

Two pointers — slow (1 step), fast (2 steps). If they meet, there's a cycle. To find cycle START: after meeting, reset one pointer to head and move both 1 step until they meet again. $O(n)$ time, $O(1)$ space.

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast: return True
    return False
```

Q41. Explain Dynamic Programming with the classic 'coin change' problem.

ADVANCED

ANSWER

DP: break problem into overlapping subproblems, store results to avoid recomputation. Two styles: top-down (memoization) or bottom-up (tabulation). Coin change: min coins to make amount n . Subproblem: $f(\text{amount}) = 1 + \min(f(\text{amount} - \text{coin}) \text{ for each coin})$.

```
def coin_change(coins, amount):
    INF = float('inf')
    dp = [INF] * (amount + 1)
    dp[0] = 0
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a:
                dp[a] = min(dp[a], dp[a - c] + 1)
    return dp[amount] if dp[amount] != INF else -1
```

■ **INTERVIEW RECIPE for DP:** (1) Define state. (2) Define recurrence. (3) Base cases. (4) Iteration order. (5) Final answer location. (6) Optimize space if possible.

Q42. Solve: longest common subsequence (LCS).

ADVANCED

ANSWER

Classic 2D DP. $dp[i][j]$ = LCS of $s1[:i]$ and $s2[:j]$. If chars match: $dp[i][j] = dp[i-1][j-1] + 1$. Else: $\max(dp[i-1][j], dp[i][j-1])$. $O(mn)$ time and space (reducible to $O(\min(m,n))$ space).

```
def lcs(s1, s2):
    m, n = len(s1), len(s2)
    dp = [[0]*(n+1) for _ in range(m+1)]
    for i in range(1, m+1):
        for j in range(1, n+1):
            if s1[i-1] == s2[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n]
```

Q43. Implement quicksort and discuss when to prefer mergesort.

ADVANCED

ANSWER

Quicksort: pick pivot, partition into $<$, $=$, $>$, recurse. Avg $O(n \log n)$, worst $O(n^2)$ on already sorted (mitigate: random pivot, median-of-three). In-place. NOT stable. Mergesort: divide and conquer, merge sorted halves. Always $O(n \log n)$, stable, but $O(n)$ extra space. Use mergesort when stability matters or for linked lists. Python's sort (Timsort) is a hybrid mergesort + insertion sort — $O(n \log n)$, stable.

```
import random
def quicksort(arr):
    if len(arr) <= 1: return arr
    pivot = random.choice(arr)
    lo = [x for x in arr if x < pivot]
    eq = [x for x in arr if x == pivot]
    hi = [x for x in arr if x > pivot]
    return quicksort(lo) + eq + quicksort(hi)
```

Q44. Topological sort — explain and implement.

ADVANCED

ANSWER

Linear ordering of DAG vertices such that for every edge $u \rightarrow v$, u comes before v . Used for build systems, course scheduling, dependency resolution. Two approaches: (1) Kahn's algorithm (BFS, in-degree); (2) DFS post-order reversed.

```
from collections import deque, defaultdict
def topo_sort(graph, n):
    indeg = [0] * n
    for u in graph:
        for v in graph[u]: indeg[v] += 1
    q = deque(i for i in range(n) if indeg[i] == 0)
    order = []
    while q:
        u = q.popleft(); order.append(u)
        for v in graph[u]:
            indeg[v] -= 1
            if indeg[v] == 0: q.append(v)
    return order if len(order) == n else [] # [] means cycle
```

4. OOP & Design Patterns

B · 3

I · 4

A · 2

Q45. Explain the four pillars of OOP.

BEGINNER

ANSWER

Encapsulation: bundling data and methods, restricting direct access (Python uses naming conventions: `_protected`, `__private` name-mangled). Inheritance: child class gets parent's attributes/methods (reuse, hierarchy). Polymorphism: same interface, different implementations (duck typing in Python). Abstraction: hide implementation behind a clean interface (abstract base classes via `abc` module).

Q46. What does `self` mean in Python?

BEGINNER

ANSWER

Conventional name for the first parameter of instance methods — the instance itself. Python passes it implicitly when you call `instance.method()`. It's just a convention — could be 'this' or anything, but DON'T deviate. Why explicit? Python's Zen: 'Explicit is better than implicit.'

Q47. What are dunder (magic) methods?

BEGINNER

ANSWER

Methods with double underscores like `__init__`, `__str__`, `__repr__`, `__len__`, `__eq__`, `__lt__`, `__hash__`, `__add__`, `__iter__`, `__call__`, `__enter__`/`__exit__`. They hook into Python's syntax — define `__add__` and 'obj + x' works. Override carefully; some have contracts (e.g. `__hash__` must be consistent with `__eq__`).

Q48. Difference between composition and inheritance — which do you prefer?

INTERMEDIATE

ANSWER

Inheritance: 'is-a' relationship, class extends another. Composition: 'has-a' relationship, class contains an instance of another. Prefer composition unless there's a true is-a relationship — inheritance creates tight coupling, makes refactoring hard, and can lead to fragile hierarchies. 'Favor composition over inheritance' — GoF.

```
# Inheritance (tight coupling)
class Car(Engine): pass      # weird — Car IS-A Engine?

# Composition (flexible)
class Car:
    def __init__(self):
        self.engine = Engine()  # Car HAS-A Engine
```

Q49. What is an abstract base class (ABC)?

INTERMEDIATE

ANSWER

A class that cannot be instantiated directly — defines an interface. Use `abc.ABC` and `@abstractmethod`. Subclasses must implement abstract methods or they too become abstract. Use for defining contracts and protocols.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self): pass

class Square(Shape):
    def __init__(self, s): self.s = s
    def area(self): return self.s ** 2

# Shape() → TypeError: abstract
Square(5).area()  # 25
```

Q50. Explain `__slots__` — when and why use it?

INTERMEDIATE

ANSWER

`__slots__` replaces the instance `__dict__` with a fixed set of attribute slots. Benefits: ~40% less memory per instance, faster attribute access, prevents accidentally adding new attributes. Costs: no instance `__dict__` (can't add ad-hoc attrs), inheritance complications. Use when creating millions of instances of small classes.

```
class Point:
    __slots__ = ('x', 'y')
    def __init__(self, x, y):
        self.x, self.y = x, y
```

```
p = Point(1, 2)
p.z = 3    # AttributeError
```

Q51. Implement the Singleton pattern in Python.

INTERMEDIATE

ANSWER

Multiple ways: (1) Override `__new__`. (2) Decorator. (3) Module-level (modules are singletons by default — most Pythonic). (4) Metaclass. For most cases, just use a module.

```
# Pythonic: just use a module!
# config.py
MAX_CONN = 100
_pool = None
def get_pool():
    global _pool
    if _pool is None: _pool = make_pool()
    return _pool

# Or via __new__:
class Singleton:
    _inst = None
    def __new__(cls):
        if cls._inst is None:
            cls._inst = super().__new__(cls)
        return cls._inst
```

Q52. Explain the Factory and Strategy patterns with Python examples.

ADVANCED

ANSWER

Factory: encapsulate object creation logic — return different subclasses based on input. Strategy: encapsulate algorithms behind a common interface, swap at runtime. In Python, both are often simplified — Strategy = a function passed as argument, Factory = a function returning instances.

```
# Factory
def notifier(kind):
    return {'email': EmailNotifier,
           'sms': SmsNotifier}[kind]()

# Strategy (just pass a function)
def sort_with(items, key_fn):
    return sorted(items, key=key_fn)

sort_with(users, key_fn=lambda u: u.age)
```

Q53. What are metaclasses? Give a real use case.

ADVANCED

ANSWER

A metaclass is the class of a class. By default, type is the metaclass of all classes. Customize class creation by defining a metaclass. 99% of Python code never needs them. Real uses: ORMs (Django Model auto-registers fields), enforcing class-level contracts, plugin registration, API frameworks. Tim Peters: 'If you wonder whether you need metaclasses, you don't.'

```
class AutoRegister(type):
    registry = {}
    def __new__(mcs, name, bases, ns):
        cls = super().__new__(mcs, name, bases, ns)
        AutoRegister.registry[name] = cls
        return cls

class Plugin(metaclass=AutoRegister): pass
class A(Plugin): pass
class B(Plugin): pass

AutoRegister.registry  # {'Plugin':..., 'A':..., 'B':...}
```

5. Decorators, Generators, Iterators

B · 1

I · 3

A · 3

Q54. What is a decorator? Give a simple example.

BEGINNER

ANSWER

A decorator is a callable that takes a function and returns a (usually modified) function. Syntactic sugar: @dec above a function is the same as func = dec(func). Use for cross-cutting concerns: logging, timing, caching, auth, retry.

```
def log(fn):
    def wrapper(*args, **kw):
        print(f'calling {fn.__name__}')
        result = fn(*args, **kw)
        print(f'returned {result}')
        return result
    return wrapper

@log
def add(a, b): return a + b

add(2, 3)
# calling add
# returned 5
```

Q55. Write a decorator that takes arguments (parameterized decorator).

INTERMEDIATE

ANSWER

Three levels of nesting: outer takes decorator args, middle takes the function, inner is the wrapper. Use functools.wraps to preserve the original function's metadata.

```
from functools import wraps

def retry(times=3):
    def deco(fn):
        @wraps(fn)
        def wrapper(*a, **kw):
            for i in range(times):
                try: return fn(*a, **kw)
                except Exception as e:
                    if i == times - 1: raise
            return wrapper
        return deco

@retry(times=5)
def flaky_api(): ...
```

■ **WHY @wraps?** Without it, your decorated function loses its `__name__`, `__doc__`, `__module__` — confusing for debuggers and IDEs.

Q56. Generators vs iterators — what's the difference?

INTERMEDIATE

ANSWER

An iterator is any object implementing `__iter__` (returns self) and `__next__` (returns next value or raises `StopIteration`). A generator is a special iterator created by a function with `yield` or a generator expression. Generators are lazy, stateful (resume where they left off), memory-efficient — never load full sequence into memory.

```
# Iterator (manual)
class Counter:
    def __init__(self, n): self.n = n; self.i = 0
    def __iter__(self): return self
    def __next__(self):
        if self.i >= self.n: raise StopIteration
        self.i += 1; return self.i

# Generator (much simpler!)
def counter(n):
    for i in range(1, n+1):
        yield i
```

Q57. When should you use a generator vs a list?

INTERMEDIATE

ANSWER

Use a generator when: (1) the sequence is large/infinite, (2) you only need to iterate once, (3) you want streaming/pipeline processing, (4) memory is constrained. Use a list when: (1) you need random access, (2) need length up front, (3) need to iterate multiple times, (4) the data is small.

```
# Memory hog
big = [x**2 for x in range(10**8)] # ~800MB

# Lazy, ~zero memory
big = (x**2 for x in range(10**8)) # generator
sum(big) # streams through
```

Q58. Explain yield from and generator delegation.

ADVANCED

ANSWER

`yield from` delegates to a sub-generator, forwarding values and exceptions. Useful for composing generators, flattening, or wrapping generators that use `send()/throw()`. Also accepts any iterable.

```
def chain(*iters):
    for it in iters:
        yield from it

list(chain([1,2], (3,4), 'ab'))
# [1, 2, 3, 4, 'a', 'b']
```


Q59. Implement a coroutine-style pipeline with generators.

ADVANCED

ANSWER

Generators can receive values via `send()` — turning them into 'coroutines'. Useful for streaming data processing pipelines. Modern Python uses `async/await` for true coroutines, but generator-based ones still appear.

```
def grep(pattern):
    print(f'searching for {pattern}')
    while True:
        line = yield
        if pattern in line:
            print(line)

g = grep('error')
next(g)          # prime
g.send('all good')
g.send('error: 500') # error: 500
```

Q60. Implement a decorator that caches results based on arguments (memoization).

ADVANCED

ANSWER

Use a dict keyed by the args. `functools.lru_cache` and `functools.cache` (3.9+) do this for you. Hand-rolled version handles only hashable args.

```
from functools import wraps

def memoize(fn):
    cache = {}
    @wraps(fn)
    def wrapper(*args):
        if args not in cache:
            cache[args] = fn(*args)
        return cache[args]
    return wrapper

@memoize
def fib(n):
    if n < 2: return n
    return fib(n-1) + fib(n-2)

# fib(100) now instant
```

6. Concurrency & Async

B · 1

I · 3

A · 2

Q61. Threading vs multiprocessing vs asyncio — when to use each?

BEGINNER

ANSWER

Threading: I/O-bound work (network, disk). GIL prevents CPU parallelism but releases on I/O. Multiprocessing: CPU-bound work — spawns separate Python processes, each with own GIL, full multi-core use. Overhead: process creation, IPC. Asyncio: I/O-bound at huge scale (thousands of connections) — cooperative single-threaded, no thread overhead, but requires async-aware libraries.

DIAGRAM

Choose by workload:

CPU-bound	→ multiprocessing
I/O-bound (few)	→ threading
I/O-bound (many)	→ asyncio
Mixed	→ asyncio + run_in_executor

Q62. What is asyncio? Walk through a simple async function.

INTERMEDIATE

ANSWER

asyncio is Python's async I/O framework. `async def` creates a coroutine; `await` pauses execution until an awaitable completes, yielding control to the event loop. Concurrency without threads — single thread, cooperative scheduling. Need async libraries (`aiohttp`, `asyncpg`) — using requests blocks the loop.

```
import asyncio

async def fetch(url):
    print(f'start {url}')
    await asyncio.sleep(1)    # simulate I/O
    print(f'done {url}')
    return url

async def main():
    results = await asyncio.gather(
        fetch('a'), fetch('b'), fetch('c'))
    print(results)

asyncio.run(main())    # all 3 fetches in ~1s, not 3s
```

Q63. What is a race condition? How do you prevent it in Python?

INTERMEDIATE

ANSWER

When two threads access shared state and the outcome depends on timing. Even with the GIL, multi-step operations aren't atomic (count += 1 = load, add, store). Prevent with threading.Lock, RLock, or atomic operations. Or, avoid shared mutable state — use queue.Queue for thread-to-thread communication.

```
import threading
lock = threading.Lock()
count = 0

def inc():
    global count
    with lock:          # critical section
        count += 1
```

Q64. Explain concurrent.futures.

INTERMEDIATE

ANSWER

High-level API for both threads and processes. ThreadPoolExecutor for I/O-bound, ProcessPoolExecutor for CPU-bound. Returns Future objects; supports map(), submit(), as_completed(). Easier than raw threading/multiprocessing for typical use.

```
from concurrent.futures import ThreadPoolExecutor
import requests

def fetch(url):
    return requests.get(url).status_code

urls = [f'https://example.com/{i}' for i in range(50)]
with ThreadPoolExecutor(max_workers=10) as ex:
    results = list(ex.map(fetch, urls))
```

Q65. What is the difference between asyncio.gather and asyncio.wait?

ADVANCED

ANSWER

gather: takes awaitables (not Tasks required), returns results in input order, by default fails fast (cancels others on first exception unless return_exceptions=True). wait: takes Tasks, returns (done, pending) sets — must inspect each, more flexible (timeout, return_when=FIRST_COMPLETED). Use gather for most cases; wait when you need fine control or staggered completion.

```
import asyncio

# Run with timeout, gather what's done
done, pending = await asyncio.wait(
    [task1, task2, task3],
    timeout=5,
    return_when=asyncio.FIRST_COMPLETED
)
for t in pending: t.cancel()
```

Q66. How do you avoid blocking the asyncio event loop with CPU-bound or blocking-I/O work?

ADVANCED

ANSWER

Use `loop.run_in_executor()` to offload to a thread pool (blocking I/O) or process pool (CPU-bound). `asyncio.to_thread()` (3.9+) is the shortcut. Never call `requests.get()`, `time.sleep()`, or heavy computation directly in async code — it freezes the loop and stalls every other coroutine.

```
import asyncio, requests

async def fetch_blocking(url):
    # WRONG: would block the loop
    # return requests.get(url)

    # RIGHT: offload to thread
    return await asyncio.to_thread(requests.get, url)
```

7. Python Internals & Gotchas

B · 2

I · 3

A · 3

Q67. Why is `def f(x=[]):` a famous Python pitfall?

BEGINNER

ANSWER

Default mutable args are evaluated ONCE at function definition, NOT every call. So `[]` is shared across all calls. Fix: use `None` as default and create inside.

```
# BUG
def append_to(x, lst=[]):
    lst.append(x); return lst
append_to(1) # [1]
append_to(2) # [1, 2] ← shared!

# FIX
def append_to(x, lst=None):
    if lst is None: lst = []
    lst.append(x); return lst
```

■ Asked in nearly every Python interview. Memorize it.

Q68. What's wrong with ``lambda i: i`` in a loop?

BEGINNER

ANSWER

Late binding: closures capture variables by reference, not by value. All lambdas see the FINAL value of `i`. Fix with default argument trick.

```
# BUG
funcs = [lambda: i for i in range(3)]
[f() for f in funcs]  # [2, 2, 2] not [0, 1, 2]!

# FIX 1: default arg captures value
funcs = [lambda i=i: i for i in range(3)]

# FIX 2: partial
from functools import partial
funcs = [partial(lambda i: i, i) for i in range(3)]
```

Q69. Explain Python's integer caching.

INTERMEDIATE

ANSWER

CPython pre-creates integer objects for -5 to 256 at startup. Any int in this range is the same object. Outside this range, integers are created on demand and may not share identity even with the same value. Don't rely on ``is`` for ints — use ``==``.

```
>>> a = 100; b = 100; a is b
True
>>> a = 1000; b = 1000; a is b
False  # different objects, same value
```

Q70. What happens with ``0.1 + 0.2``?

INTERMEDIATE

ANSWER

Returns 0.30000000000000004, not 0.3. IEEE 754 floats can't exactly represent 0.1 or 0.2. Don't compare floats with ``==``. Use `math.isclose()` or `decimal.Decimal` for exact decimal arithmetic.

```
from decimal import Decimal
Decimal('0.1') + Decimal('0.2')  # Decimal('0.3') exact

import math
math.isclose(0.1 + 0.2, 0.3)  # True
```

Q71. Explain Python's name resolution: LEGB.

INTERMEDIATE

ANSWER

Local → Enclosing → Global → Built-in. Python looks up names in this order. `global` declares assignment writes to module scope; `nonlocal` (3.x) writes to nearest enclosing function scope. Without these, `x = ...` in a function creates a new LOCAL name.

```
x = 'global'
def outer():
    x = 'outer'
    def inner():
        nonlocal x      # write to outer's x
        x = 'changed'
    inner()
    print(x)            # 'changed'
outer()
print(x)               # 'global'
```

Q72. What's the difference between `__getattr__` and `__getattribute__`?

ADVANCED

ANSWER

`__getattribute__` is called for EVERY attribute access (dangerous — easy to infinite loop). `__getattr__` is called only when normal lookup FAILS. Use `__getattr__` for default values or lazy attributes. Override `__getattribute__` only when you really need to intercept all access (proxies, logging).

```
class Lazy:
    def __getattr__(self, name):      # only on miss
        print(f'computing {name}')
        value = name.upper()
        setattr(self, name, value)   # cache
        return value

o = Lazy()
o.hello    # computing hello → 'HELLO'
o.hello    # 'HELLO' (cached, no print)
```

Q73. How does Python's `import` system work?

ADVANCED

ANSWER

When you import X: (1) check `sys.modules` cache; (2) if not found, finders/loaders in `sys.meta_path` search `sys.path`; (3) load `.py` or `.pyc`, compile to bytecode, execute module-level code; (4) bind module object to X. Circular imports break this because module is partially initialized when re-entered. Use lazy imports inside functions to defer.

Q74. Explain the difference between ``bytes``, ``str``, and ``bytearray``.

ADVANCED

ANSWER

`str`: immutable sequence of Unicode code points. `bytes`: immutable sequence of 8-bit values (0-255). `bytearray`: mutable bytes. Encoding converts `str` → `bytes` (`'hello'.encode('utf-8')`); decoding goes the other way. Always specify encoding explicitly. Python 3 separates them strictly to avoid Python 2's text/binary confusion.

8. Exceptions & Error Handling

B · 2

I · 3

A · 1

Q75. Explain `try` / `except` / `else` / `finally`.

BEGINNER

ANSWER

`try`: code that might raise. `except`: handles specific exceptions. `else`: runs ONLY if `try` succeeded (no exception). `finally`: runs ALWAYS — even on return or unhandled exception. Use `finally` for cleanup; `else` for code that should run only on success but shouldn't be 'protected' by the `try`.

```
try:
    x = compute()
except ValueError as e:
    log(e); raise
else:
    use(x)          # only if no exception
finally:
    cleanup()       # always
```

Q76. Why is ``except:`` bare-clause dangerous?

BEGINNER

ANSWER

It catches EVERYTHING including `SystemExit`, `KeyboardInterrupt`, `MemoryError` — making your program impossible to kill with `Ctrl+C` and masking critical errors. Always catch the specific exception, or ``except Exception:`` if you really need broad catch (excludes `BaseException`-only stuff like `SystemExit`).

Q77. EAFP vs LBYL — explain.

INTERMEDIATE

ANSWER

EAFP: 'Easier to Ask Forgiveness than Permission' — try the operation, catch the exception. LBYL: 'Look Before You Leap' — check conditions first. Python idiom prefers EAFP because: (1) avoids race conditions (TOCTOU), (2) often faster (happy path no overhead). Exceptions: when the exception path is common, LBYL may be clearer.

```
# LBYL
if 'key' in d: x = d['key']

# EAFP (more Pythonic)
try: x = d['key']
except KeyError: ...
```

Q78. How do you create a custom exception?

INTERMEDIATE

ANSWER

Subclass Exception (or a more specific built-in). Add attributes if you need context. Use a hierarchy for your library so callers can catch broadly or specifically.

```
class APIError(Exception):
    """Base for all API errors."""

class NotFoundError(APIError): pass
class AuthError(APIError):
    def __init__(self, user):
        super().__init__(f'auth failed for {user}')
        self.user = user

try: ...
except AuthError as e: print(e.user)
except APIError: ...
```

Q79. What is exception chaining (`raise X from Y`)?

INTERMEDIATE

ANSWER

When you catch an exception and raise a different one, Python tracks both. `raise NewErr from old` sets `\_\_cause\_\_` (explicit chain). Implicit chaining sets `\_\_context\_\_`. The traceback shows 'The above exception was the direct cause of...'. Use 'from None' to suppress chaining.

```
try:
    open('missing')
except FileNotFoundError as e:
    raise ConfigError('cannot load') from e
```


Q80. What are exception groups (Python 3.11+)?

ADVANCED

ANSWER

ExceptionGroup wraps multiple exceptions that occurred in parallel (e.g. `asyncio.TaskGroup`). New `except*` syntax filters by type, catching subset. Powerful for structured concurrency where multiple sibling tasks may fail.

```
try:
    raise ExceptionGroup('errs', [
        ValueError('a'), KeyError('b')
    ])
except* ValueError as eg:
    print('values:', eg.exceptions)
except* KeyError as eg:
    print('keys:', eg.exceptions)
```

9. Strings & Regex

B · 2

I · 3

A · 1

Q81. List 10 string methods you use most often.

BEGINNER

ANSWER

`split()`, `join()`, `strip()/lstrip()/rstrip()`, `replace()`, `startswith()/endswith()`, `find()/index()` (`find` returns `-1`, `index` raises), `lower()/upper()`, `format()` / f-strings, `count()`, `isdigit()/isalpha()/isalnum()`. For performance-critical concatenation in loops, prefer `".join(list)"` over `+=` in a loop.

Q82. f-strings vs `.format()` vs `%` formatting — which to use?

BEGINNER

ANSWER

f-strings (Python 3.6+) are fastest and most readable. Use them. `.format()` is still useful when the template comes from outside (config file). `%` is legacy — avoid except for logging (lazy formatting).

```
name, age = 'Bob', 30
f'{name} is {age}'           # best
'{} is {}'.format(name, age)
'%s is %d' % (name, age)

# Logging exception: % defers formatting
log.debug('user %s logged in', name) # only formats if level enabled
```

Q83. How would you reverse a string and reverse the words in a sentence?

INTERMEDIATE

ANSWER

Reverse string: `s[::-1]` (slice with step -1). Reverse word order: `' '.join(s.split()[::-1])`. Reverse each word in place: `' '.join(w[::-1] for w in s.split())`.

```
s = 'hello world'
s[::-1]                # 'dlrow olleh'
' '.join(s.split()[::-1]) # 'world hello'
' '.join(w[::-1] for w in s.split()) # 'olleh dlrow'
```

Q84. Check if two strings are anagrams.

INTERMEDIATE

ANSWER

Two approaches: (1) sort and compare — $O(n \log n)$. (2) Counter — $O(n)$. Counter is preferred. Variations: handle case-insensitive, ignore spaces, Unicode normalization.

```
from collections import Counter

def anagram(a, b):
    return Counter(a) == Counter(b)

# Or:
def anagram2(a, b):
    return sorted(a) == sorted(b)
```

Q85. Explain the basic re module operations.

INTERMEDIATE

ANSWER

`re.search`: find first match anywhere. `re.match`: match only at start. `re.fullmatch`: must match the whole string. `re.findall`: list of all matches. `re.finditer`: iterator of Match objects. `re.sub`: replace. Precompile patterns with `re.compile()` if reused. Use raw strings (`r'...'`) for patterns to avoid escape hell.

```
import re
text = 'errors: 404, 500, 503'
re.findall(r'\d+', text)      # ['404', '500', '503']
re.sub(r'\d+', 'XXX', text)   # 'errors: XXX, XXX, XXX'

pat = re.compile(r'(\w+)@(\w+)')
m = pat.search('hi bob@x.com')
m.group(0)    # 'bob@x'
m.group(1)    # 'bob'
m.group(2)    # 'x'
```

Q86. Why can `+=` on strings in a loop be $O(n^2)$?

ADVANCED

ANSWER

Strings are immutable — each `+=` creates a new string and copies the old content. n appends = total work proportional to $1 + 2 + \dots + n = O(n^2)$. CPython has an optimization for the trivial 's += x' case but don't rely on it. ALWAYS use `".join(list)`.

```
# SLOW:  $O(n^2)$ 
s = ''
for w in words: s += w

# FAST:  $O(n)$ 
s = ''.join(words)

# Or io.StringIO for building chunks
```

10. FAANG Coding Problems

A · 6

Q87. Best Time to Buy and Sell Stock (one transaction).

INTERMEDIATE

ANSWER

Track minimum so far; at each step compute profit if selling today. Single pass $O(n)$ time, $O(1)$ space.

```
def max_profit(prices):
    min_price = float('inf')
    best = 0
    for p in prices:
        min_price = min(min_price, p)
        best = max(best, p - min_price)
    return best
```

Q88. Group Anagrams.

INTERMEDIATE

ANSWER

Hash each word by its sorted characters (or character count tuple). Group into a dict.

```
from collections import defaultdict
def group_anagrams(words):
    groups = defaultdict(list)
    for w in words:
        key = ''.join(sorted(w))
        groups[key].append(w)
    return list(groups.values())

# ['eat', 'tea', 'tan', 'ate', 'nat', 'bat']
# → [['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
```

Q89. Valid Parentheses.

INTERMEDIATE

ANSWER

Stack. Push opening, pop and check on closing. Empty stack at end = valid.

```
def is_valid(s):
    stack = []
    pairs = {'(': ')', '[': ']', '{': '}'
    for c in s:
        if c in '([{':
            stack.append(c)
        else:
            if not stack or stack.pop() != pairs[c]:
                return False
    return not stack
```

Q90. Merge Intervals.

INTERMEDIATE

ANSWER

Sort by start. Scan: if current overlaps with the last in result, extend; else append.

```
def merge(intervals):
    intervals.sort(key=lambda x: x[0])
    out = []
    for start, end in intervals:
        if out and start <= out[-1][1]:
            out[-1][1] = max(out[-1][1], end)
        else:
            out.append([start, end])
    return out
```

Q91. Kth Largest Element in an array.

INTERMEDIATE

ANSWER

Three approaches: (1) sort $O(n \log n)$. (2) min-heap of size k — $O(n \log k)$. (3) Quickselect — average $O(n)$. `heapq.nlargest` is the easy answer.

```
import heapq
def kth_largest(nums, k):
    return heapq.nlargest(k, nums)[-1]

# Or:
def kth(nums, k):
    h = []
    for n in nums:
        heapq.heappush(h, n)
        if len(h) > k: heapq.heappop(h)
    return h[0]
```

Q92. Word Break — given a string *s* and a dict, can *s* be split into dict words?

ADVANCED

ANSWER

DP: $dp[i] = \text{True}$ iff $s[:i]$ is breakable. $dp[i] = \text{any}(dp[j] \text{ and } s[j:i] \text{ in words for } j < i)$. $O(n^2)$.

```
def word_break(s, words):
    words = set(words)
    n = len(s)
    dp = [False] * (n + 1)
    dp[0] = True
    for i in range(1, n + 1):
        for j in range(i):
            if dp[j] and s[j:i] in words:
                dp[i] = True
                break
    return dp[n]
```

Q93. Number of Islands (DFS/BFS on grid).

ADVANCED

ANSWER

For each unvisited land cell, do DFS/BFS marking all connected land. Count starts.

```
def num_islands(grid):
    if not grid: return 0
    rows, cols = len(grid), len(grid[0])
    count = 0
    def dfs(r, c):
        if not (0 <= r < rows and 0 <= c < cols) or grid[r][c] != '1':
            return
        grid[r][c] = '0' # mark visited
        for dr, dc in [(1,0), (-1,0), (0,1), (0,-1)]:
            dfs(r+dr, c+dc)
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                count += 1
                dfs(r, c)
    return count
```

Q94. Longest Substring Without Repeating Characters.

ADVANCED

ANSWER

Sliding window with a map of char → last position. When you see a repeat inside the window, jump left to right of previous occurrence.

```
def longest(s):
    last = {}
    l = 0; best = 0
    for r, c in enumerate(s):
        if c in last and last[c] >= l:
            l = last[c] + 1
        last[c] = r
        best = max(best, r - l + 1)
    return best
```

Q95. LRU Cache design (LeetCode 146).

ADVANCED

ANSWER

OrderedDict gives O(1) get/put with move_to_end. Or hand-roll: dict + doubly linked list. Implemented earlier in the Data Structures section — same solution.

Q96. Trapping Rain Water.

ADVANCED

ANSWER

Two pointers from each end; track max_left and max_right; water at each cell = min(max_left, max_right) - height[i]. O(n) time, O(1) space.

```
def trap(height):
    l, r = 0, len(height) - 1
    ml = mr = water = 0
    while l < r:
        if height[l] < height[r]:
            ml = max(ml, height[l])
            water += ml - height[l]
            l += 1
        else:
            mr = max(mr, height[r])
            water += mr - height[r]
            r -= 1
    return water
```

Q97. Serialize and Deserialize a Binary Tree.

ADVANCED

ANSWER

Preorder with null markers. Serialize via DFS recording values and 'N' for nulls. Deserialize by consuming tokens left-to-right.

```
class TreeNode:
    def __init__(self, v, l=None, r=None): self.val, self.left, self.right = v, l, r

def serialize(root):
    out = []
    def dfs(n):
        if not n: out.append('N'); return
        out.append(str(n.val))
        dfs(n.left); dfs(n.right)
    dfs(root)
    return ','.join(out)

def deserialize(s):
    it = iter(s.split(','))
    def build():
        v = next(it)
        if v == 'N': return None
        return TreeNode(int(v), build(), build())
    return build()
```

11. Pythonic Idioms & Best Practices

B · 2

I · 2

A · 2

Q98. What is 'Pythonic' code? Give 5 examples.

BEGINNER

ANSWER

Code that uses language idioms naturally. Examples: (1) `for x in seq` not `for i in range(len(seq))`. (2) `if not seq` not `if len(seq) == 0`. (3) tuple unpacking `a, b = b, a`. (4) list comprehensions over manual loops. (5) `with open()` over manual try/finally. (6) `enumerate()` for index+value. (7) `zip()` for parallel iteration.

Q99. List comprehension vs generator expression vs map/filter.

BEGINNER

ANSWER

List comp: `[x*2 for x in nums]` — eager, creates list. Gen expr: `(x*2 for x in nums)` — lazy iterator. map/filter: functional, returns iterators in Python 3. Prefer list comp/gen expr for readability; map/filter only when you have an existing function and no transformation logic.

```
# All equivalent, but readability wins
result = [f(x) for x in xs if cond(x)]
result = list(map(f, filter(cond, xs))) # less readable
```

Q100. Show 10 useful Python idioms you'd put on a cheat sheet.

INTERMEDIATE

ANSWER

(1) ``a, *rest = [1,2,3,4]`` — extended unpacking. (2) ``dict.get(k, default)``. (3) ``dict.setdefault(k, [])``. (4) ``collections.defaultdict(list)``. (5) ``enumerate(seq, start=1)``. (6) ``zip(*matrix)`` to transpose. (7) ``sorted(items, key=..., reverse=True)``. (8) ``any()`` / ``all()``. (9) ``from itertools import chain, groupby, islice``. (10) ``*`` to enforce keyword-only args.

```
# Transpose
m = [[1,2,3],[4,5,6]]
list(zip(*m))      # [(1,4),(2,5),(3,6)]

# Group consecutive
from itertools import groupby
[list(g) for k, g in groupby('AAABBC')]
# [['A','A','A'],['B','B'],['C']]
```

Q101. What is the walrus operator (:=)?

INTERMEDIATE

ANSWER

Assignment expression (Python 3.8+). Assigns and returns a value in one expression. Useful in while loops, comprehensions, and to avoid recomputation. Don't overuse — can hurt readability.

```
# Without walrus
while True:
    chunk = f.read(1024)
    if not chunk: break
    process(chunk)

# With walrus
while chunk := f.read(1024):
    process(chunk)

# In comprehension: filter then transform
[y for x in data if (y := expensive(x)) > 0]
```


Q102. What is type hinting? Should you use it?

ADVANCED

ANSWER

Annotations describing expected types (PEP 484). Not enforced at runtime — checked by tools (mypy, pyright). Pros: catches bugs early, better IDE support, self-documenting. Cons: maintenance overhead, can become verbose. Use for: library APIs, large codebases, anything > 100 lines. Skip for quick scripts.

```
from typing import Optional, Union

def parse(s: str, default: Optional[int] = None) -> int:
    try:
        return int(s)
    except ValueError:
        if default is None: raise
        return default

# Modern (3.10+):
def parse(s: str, default: int | None = None) -> int: ...
```

Q103. What are dataclasses? When use them vs namedtuple vs Pydantic?

ADVANCED

ANSWER

dataclasses (3.7+): auto-generate `__init__`, `__repr__`, `__eq__` from class annotations. Mutable by default; `frozen=True` for immutable. namedtuple: lightweight, immutable, tuple-like. Use for fixed-position records. Pydantic: runtime validation, JSON parsing, settings — heavier but rich. Rule: dataclass for simple data classes, namedtuple for ultra-light immutables, Pydantic for I/O boundaries.

```
from dataclasses import dataclass, field

@dataclass
class User:
    name: str
    age: int = 0
    tags: list = field(default_factory=list)

u = User('alice', 30)
u.tags.append('admin')
```

12. Python in System Design

A · 3

Q104. How would you implement rate limiting in Python?

INTERMEDIATE

ANSWER

Token bucket: refill at fixed rate up to a max, each request consumes one. Sliding window: count requests in last N seconds. For single process — in-memory dict. For distributed — Redis with atomic ops (INCR + EXPIRE, or Lua script for sliding window).

```
import time

class TokenBucket:
    def __init__(self, rate, capacity):
        self.rate = rate; self.capacity = capacity
        self.tokens = capacity; self.last = time.time()
    def allow(self):
        now = time.time()
        self.tokens = min(self.capacity,
            self.tokens + (now - self.last) * self.rate)
        self.last = now
        if self.tokens >= 1:
            self.tokens -= 1; return True
        return False
```

Q105. Design a thread-safe in-memory cache with TTL.

INTERMEDIATE

ANSWER

Dict storing (value, expiry) tuples. Lock for thread safety. Periodic cleanup or lazy expiration on access.

```
import time, threading

class TTLCache:
    def __init__(self): self.d = {}; self.lock = threading.Lock()
    def set(self, k, v, ttl):
        with self.lock:
            self.d[k] = (v, time.time() + ttl)
    def get(self, k):
        with self.lock:
            if k not in self.d: return None
            v, exp = self.d[k]
            if time.time() > exp:
                del self.d[k]; return None
            return v
```

Q106. How would you build a simple producer-consumer pipeline?

ADVANCED

ANSWER

queue.Queue (thread-safe) between producer and consumer threads. Use sentinel (None) to signal shutdown. For high-throughput, multiprocessing.Queue or asyncio.Queue. For very large scale — Kafka, RabbitMQ.

```
import threading, queue

q = queue.Queue(maxsize=100)
SENTINEL = object()

def producer():
    for i in range(1000): q.put(i)
    q.put(SENTINEL)

def consumer():
    while True:
        item = q.get()
        if item is SENTINEL: break
        process(item)
        q.task_done()

threading.Thread(target=producer).start()
threading.Thread(target=consumer).start()
```

Q107. How do you scale a Python web service?

ADVANCED

ANSWER

(1) Run multiple workers (gunicorn -w N, uvicorn workers). Python's GIL means single-process throughput is limited — workers give CPU parallelism. (2) Use async framework (FastAPI/aiohttp) for I/O-bound. (3) Put cache (Redis) in front. (4) Database connection pooling. (5) Horizontal scale behind a load balancer. (6) Offload heavy work to background queues (Celery, RQ, dramatiq). (7) Profile with py-spy or cProfile before optimizing.

Q108. What's the difference between WSGI and ASGI?

ADVANCED

ANSWER

WSGI: synchronous, one request per worker thread (Flask, Django). ASGI: async-capable, supports HTTP + WebSocket, lifespan events (FastAPI, Starlette, Django Channels). For modern apps with WebSockets, SSE, or massive concurrency, choose ASGI. For traditional REST APIs, WSGI is still totally fine — and simpler.

13. Testing & Debugging

B · 2

I · 3

A · 2

Q109. What testing frameworks do you use in Python?

BEGINNER

ANSWER

pytest (de facto standard — simple, plugin ecosystem, fixtures, parametrize), unittest (stdlib — class-based, Java-style), doctest (tests in docstrings). pytest wins for most modern projects. Use pytest with coverage.py for coverage reports, hypothesis for property-based testing.

Q110. Write a simple pytest test.

BEGINNER

ANSWER

Just a function starting with test_. Use plain assert — pytest rewrites it to show diffs. Fixtures via @pytest.fixture decorator. Parametrize with @pytest.mark.parametrize.

```
import pytest

def add(a, b): return a + b

def test_add():
    assert add(2, 3) == 5

@pytest.mark.parametrize('a,b,expected', [
    (1, 1, 2), (0, 0, 0), (-1, 1, 0)
])
def test_add_param(a, b, expected):
    assert add(a, b) == expected
```

Q111. What are fixtures and how do they work in pytest?

INTERMEDIATE

ANSWER

Fixtures provide setup/teardown and shared test data. Defined with @pytest.fixture. Test functions request them by parameter name. Scopes: function (default), class, module, session — control reuse. yield-based fixtures support cleanup after yield.

```
@pytest.fixture
def db():
    conn = create_test_db()
    yield conn
    conn.close()          # teardown

def test_user(db):
    db.insert(...)
    assert db.query(...)
```

Q112. How do you mock dependencies in tests?

INTERMEDIATE

ANSWER

unittest.mock (or pytest-mock). patch the dependency at its USE site, not where it's defined. Use MagicMock for objects, return_value/side_effect to control behavior, assert_called_with to verify calls. For HTTP, prefer responses or httpx_mock over deep mocking.

```
from unittest.mock import patch, MagicMock

# Patch where it's USED
@patch('myapp.module.requests.get')
def test_fetch(mock_get):
    mock_get.return_value.json.return_value = {'ok': True}
    result = fetch_data()
    mock_get.assert_called_with('https://api.com')
    assert result == {'ok': True}
```

■ **GOTCHA:** patch where the dependency is **LOOKED UP**, not where it lives. If module_b imports requests, patch 'module_b.requests' not 'requests'.

Q113. How do you debug Python? What tools have you used?

INTERMEDIATE

ANSWER

pdb (built-in), ipdb (pdb with IPython), breakpoint() (built-in in 3.7+, calls pdb by default but configurable via PYTHONBREAKPOINT). For production: py-spy (sampling profiler, no code change, can attach), tracemalloc (memory leaks), cProfile + snakeviz (profile visualization), logging with levels. IDE debuggers (PyCharm, VS Code) for stepping.

```
# Modern way
def buggy():
    x = compute()
    breakpoint()          # drops you into pdb
    return x
```

Q114. How do you profile a slow Python script?

ADVANCED

ANSWER

(1) cProfile for function-level timing: python -m cProfile -o out.prof script.py, then visualize with snakeviz. (2) line_profiler for line-by-line: @profile decorator + kernprof. (3) py-spy for production sampling profiling — no code change. (4) memory_profiler / tracemalloc for memory. Always profile BEFORE optimizing — intuition lies.

```
import cProfile, pstats

with cProfile.Profile() as pr:
    slow_function()

stats = pstats.Stats(pr).sort_stats('cumulative')
stats.print_stats(20)  # top 20
```

Q115. What is property-based testing?

ADVANCED

ANSWER

Instead of hand-picking examples, declare properties that should hold and let the library generate hundreds of random inputs to find counterexamples. Tool: hypothesis. Catches edge cases humans miss (empty inputs, unicode, huge numbers). Shrinks failing examples to minimal form.

```
from hypothesis import given, strategies as st

@given(st.lists(st.integers()))
def test_sort_idempotent(xs):
    assert sorted(sorted(xs)) == sorted(xs)

@given(st.text())
def test_roundtrip(s):
    assert s.encode('utf-8').decode('utf-8') == s
```

14. Standard Library Essentials

B · 1

I · 3

A · 2

Q116. Name 10 standard library modules every Python developer should know.

BEGINNER

ANSWER

os/pathlib (filesystem), sys (interpreter), json (serialization), datetime (dates/times), collections (data structures), itertools (iterators), functools (functional tools), re (regex), logging (structured logging), unittest/pytest (testing). Plus: subprocess (shell), argparse (CLIs), csv, sqlite3, urllib/http, hashlib, pickle, threading, asyncio, multiprocessing.

Q117. Show 5 powerful itertools recipes.

INTERMEDIATE

ANSWER

itertools is full of efficient iterator combinators. Most common: chain (concatenate iterables), product (cartesian), permutations, combinations, groupby (group consecutive), accumulate (running totals), islice (slice an iterator), tee (duplicate an iterator), cycle (infinite repeat), zip_longest (pad short iterables).

```
from itertools import *

list(chain([1,2], [3,4]))          # [1,2,3,4]
list(product([1,2], [3,4]))        # [(1,3), (1,4), (2,3), (2,4)]
list(combinations('abc', 2))       # [('a','b'), ('a','c'), ('b','c')]
list(accumulate([1,2,3,4]))        # [1,3,6,10]
list(islice(count(), 5))           # [0,1,2,3,4]
```

Q118. What does functools provide?

INTERMEDIATE

ANSWER

lru_cache/cache (memoization), partial (pre-fill arguments), reduce, wraps (preserves decorated function's metadata), singledispatch (overload by type), cached_property (cache property result on instance), total_ordering (auto-generate comparison methods from `__eq__` + one of `__lt__`/`__le__`/`__gt__`/`__ge__`).

```
from functools import lru_cache, partial, cached_property

@lru_cache(maxsize=128)
def fib(n):
    return n if n < 2 else fib(n-1) + fib(n-2)

add = partial(lambda a, b: a + b, 10)
add(5)    # 15

class Lazy:
    @cached_property
    def heavy(self):
        return expensive_calc()
```

Q119. How do you handle dates and times in Python?

INTERMEDIATE

ANSWER

datetime module: datetime, date, time, timedelta, timezone. Always use timezone-aware datetimes for anything user-facing — naive datetimes are a bug factory. Use ZoneInfo (3.9+) over pytz. For parsing, datetime.fromisoformat() and strptime. For relative deltas (months, years), use dateutil.relativedelta.

```
from datetime import datetime, timezone, timedelta
from zoneinfo import ZoneInfo

now = datetime.now(timezone.utc)
later = now + timedelta(hours=2)

local = now.astimezone(ZoneInfo('America/New_York'))
parsed = datetime.fromisoformat('2026-01-15T10:30:00+00:00')
```

Q120. json vs pickle vs MessagePack — when to use each?

ADVANCED

ANSWER

json: human-readable, language-agnostic, safe to deserialize from untrusted sources. Use for APIs and config. pickle: Python-only, can serialize almost any Python object (functions, classes). DANGEROUS — never unpickle untrusted data (arbitrary code execution). Fast for Python-to-Python. MessagePack/Protocol Buffers/Avro: compact binary, faster than JSON, language-agnostic, but require schemas. Use for high-throughput RPC.

Q121. How does logging work and how should you configure it for a production app?

ADVANCED

ANSWER

logging module: Loggers (hierarchy by name), Handlers (where logs go — file, stream, syslog), Formatters (how they look), Filters (selective output). Best practices: (1) Get logger by `__name__` in each module. (2) Configure once at app startup via `dictConfig`. (3) Use structured logging (JSON) in prod for searchability. (4) Don't use root logger directly. (5) Pass `extra={}` for contextual data. (6) Use `%s` formatting style for performance (lazy evaluation).

```
import logging
logger = logging.getLogger(__name__)

# Lazy formatting – string built ONLY if level enabled
logger.debug('user %s did %s', user_id, action)

# With context
logger.info('order placed', extra={'order_id': oid, 'amount': amt})
```

15. Company-Favorite Questions

A · 7

Q122. [Google] Reverse words in a string in place (no extra space).

INTERMEDIATE

ANSWER

True in-place isn't possible for Python strings (immutable). Convert to list. Two-step: reverse whole string, then reverse each word. $O(n)$ time.

```
def reverse_words(s):
    a = list(s)
    def rev(l, r):
        while l < r: a[l], a[r] = a[r], a[l]; l+=1; r-=1
    rev(0, len(a)-1)
    start = 0
    for i in range(len(a)+1):
        if i == len(a) or a[i] == ' ':
            rev(start, i-1); start = i + 1
    return ''.join(a)

# 'the sky is blue' → 'blue is sky the'
```


Q123. [Amazon] Design a Logger Rate Limiter (log message at most once per 10 seconds).

INTERMEDIATE

ANSWER

Map message → next allowed timestamp. shouldPrintMessage(timestamp, message) returns True if allowed, updates map.

```
class Logger:
    def __init__(self):
        self.last = {}
    def shouldPrintMessage(self, ts, msg):
        if msg not in self.last or ts >= self.last[msg]:
            self.last[msg] = ts + 10
            return True
        return False
```

Q124. [Meta] Subarray Sum Equals K.

INTERMEDIATE

ANSWER

Prefix sums + hash map: count cumulative sums; for each sum S, the number of subarrays ending here with target sum k = count of (S - k) seen so far. O(n).

```
def subarray_sum(nums, k):
    count = 0; s = 0
    prev = {0: 1}
    for n in nums:
        s += n
        count += prev.get(s - k, 0)
        prev[s] = prev.get(s, 0) + 1
    return count
```

Q125. [Apple] Find the missing number in [0..n] given an array of n distinct numbers.

INTERMEDIATE

ANSWER

Three approaches: (1) sum: $n*(n+1)/2 - \text{sum}(\text{nums})$. (2) XOR all indices and all numbers — survivor is missing. (3) Sort then scan. Sum is cleanest.

```
def missing(nums):
    n = len(nums)
    return n * (n + 1) // 2 - sum(nums)

# XOR approach (overflow-safe in other langs)
def missing_xor(nums):
    result = len(nums)
    for i, n in enumerate(nums):
        result ^= i ^ n
    return result
```

Q126. [Netflix] Implement a Producer-Consumer with bounded queue.

INTERMEDIATE

ANSWER

Already covered in System Design section — queue.Queue with maxsize blocks producer when full, blocks consumer when empty. Sentinel for shutdown.

Q127. [Google] Median of Two Sorted Arrays in $O(\log \min(m,n))$.

ADVANCED

ANSWER

Binary search on the smaller array to find a partition such that left halves of both arrays form the lower half of the merged sorted array. Hard problem — practice the partitioning invariants.

```
def find_median(a, b):
    if len(a) > len(b): a, b = b, a
    m, n = len(a), len(b)
    lo, hi = 0, m
    while lo <= hi:
        i = (lo + hi) // 2
        j = (m + n + 1) // 2 - i
        a_left = a[i-1] if i > 0 else float('-inf')
        a_right = a[i] if i < m else float('inf')
        b_left = b[j-1] if j > 0 else float('-inf')
        b_right = b[j] if j < n else float('inf')
        if a_left <= b_right and b_left <= a_right:
            if (m + n) % 2 == 0:
                return (max(a_left, b_left) + min(a_right, b_right)) / 2
            return max(a_left, b_left)
        elif a_left > b_right: hi = i - 1
        else: lo = i + 1
```

Q128. [Amazon] Design Twitter — show recent tweets from followed users.**ADVANCED****ANSWER**

Users follow others; each user has a tweet list. `getNewsFeed` merges latest tweets from all followed users. Use a max-heap by timestamp across all followed users' tweet lists for an efficient k-way merge. $O(N \log k)$ where k = number followed.

```
from collections import defaultdict
import heapq
import itertools

class Twitter:
    def __init__(self):
        self.tweets = defaultdict(list)      # user → [(ts, tid)]
        self.following = defaultdict(set)
        self.ts = itertools.count()
    def postTweet(self, uid, tid):
        self.tweets[uid].append((next(self.ts), tid))
    def getNewsFeed(self, uid):
        users = self.following[uid] | {uid}
        all_tweets = [t for u in users for t in self.tweets[u]]
        return [tid for _, tid in heapq.nlargest(10, all_tweets)]
    def follow(self, follower, followee):
        self.following[follower].add(followee)
    def unfollow(self, follower, followee):
        self.following[follower].discard(followee)
```

Q129. [Meta] LRU Cache with $O(1)$ get and put using `OrderedDict` — already covered. Discuss extending to LFU.**ADVANCED****ANSWER**

LFU (Least Frequently Used) is trickier. Need: dict for key→value, dict for key→freq, dict for freq→`OrderedDict` of keys (for tie-break by recency within same freq), and a `min_freq` pointer. On access, bump freq and move key to higher-freq bucket. On evict, remove from `min_freq` bucket. $O(1)$ for all ops.

Q130. [Cisco] How would you parse very large JSON file without loading into memory?**ADVANCED****ANSWER**

`json.load` loads the whole file. For streaming: use `ijson` — generator-based, yields events or items as it parses. Or `jq` from shell, or `jsonlines` format (one JSON object per line) which you can parse line-by-line with `json.loads`.

```
import ijson

with open('huge.json', 'rb') as f:
    for record in ijson.items(f, 'records.item'):
        process(record)    # streams, constant memory

# jsonlines: simpler if you control format
with open('data.jsonl') as f:
    for line in f:
        record = json.loads(line)
```

Q131. [Zoom] Find the longest palindromic substring.

ADVANCED

ANSWER

Two approaches: (1) Expand around each center — $O(n^2)$ time, $O(1)$ space — the standard interview answer. (2) Manacher's algorithm — $O(n)$ but complex; rarely required. Cover odd and even length centers.

```
def longest_palindrome(s):
    if not s: return ''
    def expand(l, r):
        while l >= 0 and r < len(s) and s[l] == s[r]:
            l -= 1; r += 1
        return s[l+1:r]
    best = ''
    for i in range(len(s)):
        for cand in (expand(i, i), expand(i, i+1)):
            if len(cand) > len(best): best = cand
    return best
```

Q132. [Oracle] Find duplicates in a list in $O(n)$ time, $O(1)$ extra space.

ADVANCED

ANSWER

If values are in range $[1..n]$ and array has length n , mark visited by negating $\text{arr}[\text{abs}(x)-1]$. When you see an already-negative entry, x is a duplicate. Mutates input.

```
def find_duplicates(nums):
    result = []
    for x in nums:
        i = abs(x) - 1
        if nums[i] < 0:
            result.append(abs(x))
        else:
            nums[i] = -nums[i]
    return result
```

Q133. [Palo Alto] Detect SQL injection patterns in a string.

ADVANCED

ANSWER

Defensive answer: don't try to detect — use parameterized queries (the only safe answer). If asked anyway: look for suspicious patterns (UNION SELECT, ' OR '1'='1, --, ;, etc.) with regex; but emphasize this is a band-aid. Real defense: prepared statements, ORM, input validation by type/format, least-privilege DB user.

■ Interviewers test whether you reach for input validation/parameterized queries FIRST. Pattern matching alone is a red flag.

16. Cheatsheets & Reference Tables

Quick-glance tables for the night before your interview.

Big-O Cheat Sheet

Data Structure	Access	Search	Insert	Delete
list (array)	$O(1)$	$O(n)$	$O(n)^*$	$O(n)$
list append	—	—	$O(1)^\dagger$	—
list pop end	—	—	—	$O(1)$
list pop front	—	—	—	$O(n)$
dict / set	—	$O(1)_{avg}$	$O(1)_{avg}$	$O(1)_{avg}$
deque	$O(n)$	$O(n)$	$O(1)$ ends	$O(1)$ ends
heap (heapq)	$O(1)$ min	$O(n)$	$O(\log n)$	$O(\log n)$
tuple	$O(1)$	$O(n)$	—	—
str	$O(1)$	$O(n)$	—	—
* insert in middle; $\dagger$ amortized				

Sorting Algorithms

Algorithm	Time avg	Time worst	Space	Stable?
Timsort (built-in)	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quicksort	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Mergesort	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Heapsort	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Insertion sort	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Bubble sort	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Counting sort	$O(n + k)$	$O(n + k)$	$O(k)$	Yes
Radix sort	$O(d \cdot (n + k))$	$O(d \cdot (n + k))$	$O(n + k)$	Yes

Algorithm Pattern Cheat Sheet

Pattern	Use When	Template
Two pointers	sorted array, palindrome, pair sum	$l=0$, $r=n-1$; move based on condition
Sliding window	contiguous subarray/substring	expand r ; shrink l while invalid
BFS	shortest path unweighted, level order	queue; visit by layers
DFS	all paths, connectivity, cycles, topo	recurse or explicit stack
Backtracking	subsets, permutations, n -queens	choose $\rightarrow$ recurse $\rightarrow$ un-choose

Pattern	Use When	Template
Binary search	sorted, or monotonic predicate	lo, hi; mid; halve
Dynamic programming	overlapping subproblems + optimal substructure	define state → recurrence → base
Greedy	local choice → global optimum	pick best at each step, prove correct
Union-Find	connected components, MST	find with path comp; union by rank
Topological sort	DAG ordering, deps	Kahn's BFS or DFS post-order
Dijkstra	shortest path, weighted ≥ 0	min-heap of (dist, node)
Trie	prefix/autocomplete/word lookup	dict children + end flag
Heap	top-k, scheduling, k-way merge	heapq.nlargest / nsmallest
Monotonic stack	next greater/smaller element	stack stores indices in mono order
Prefix sum	range sum queries	psum[i] = psum[i-1] + a[i]

Python Trick Notes (Most-Asked Gotchas)

Gotcha	What Happens	Fix
def f(x=[]): mutable default	[] shared across calls	use None default
lambda i: i in loop	all capture final i	lambda i=i: i
a = b = []	both same list	a = []; b = []
0.1 + 0.2 != 0.3	float imprecision	math.isclose or Decimal
'1' + 1	TypeError	int('1') + 1
a is b for ints	only -5..256 cached	use ==
d.keys() during iteration	RuntimeError if you modify d	list(d.keys()) first
copy.copy on nested	inner refs shared	copy.deepcopy
class Foo: bar = []	class-level shared across inst.	init in __init__
== vs is	is = identity, == = value	use is only for None/bool
range(len(seq))	not Pythonic	enumerate(seq)
except: bare	catches SystemExit too	except Exception:
== on dict views	set-like for keys/items	expected behavior
string += in loop	$O(n^2)$	".join(list)
global in funcs	needed only to ASSIGN	for reads, no global needed

Common String/List Operations

Operation	Code	Complexity
Reverse string	s[::-1]	$O(n)$
Check palindrome	s == s[::-1]	$O(n)$

Operation	Code	Complexity
Char frequency	Counter(s)	O(n)
Sort string chars	".join(sorted(s))	O(n log n)
Reverse list in place	lst.reverse()	O(n)
Sort list	lst.sort() / sorted(lst)	O(n log n)
Sort by key desc	sorted(lst, key=fn, reverse=True)	O(n log n)
Remove duplicates (keep order)	list(dict.fromkeys(lst))	O(n)
Flatten one level	[x for sub in lst for x in sub]	O(n)
Transpose matrix	list(zip(*matrix))	O(mn)
Group dict by key	defaultdict(list)	O(n)
Max with tie-break	max(lst, key=lambda x: (x.a, -x.b))	O(n)
Top-k smallest	heapq.nsmallest(k, lst)	O(n log k)
Binary search insert point	bisect.bisect_left(arr, x)	O(log n)
Powerset	chain.from_iterable(combinations(s,r) for r in range(len(s)+1))	O(2^n)